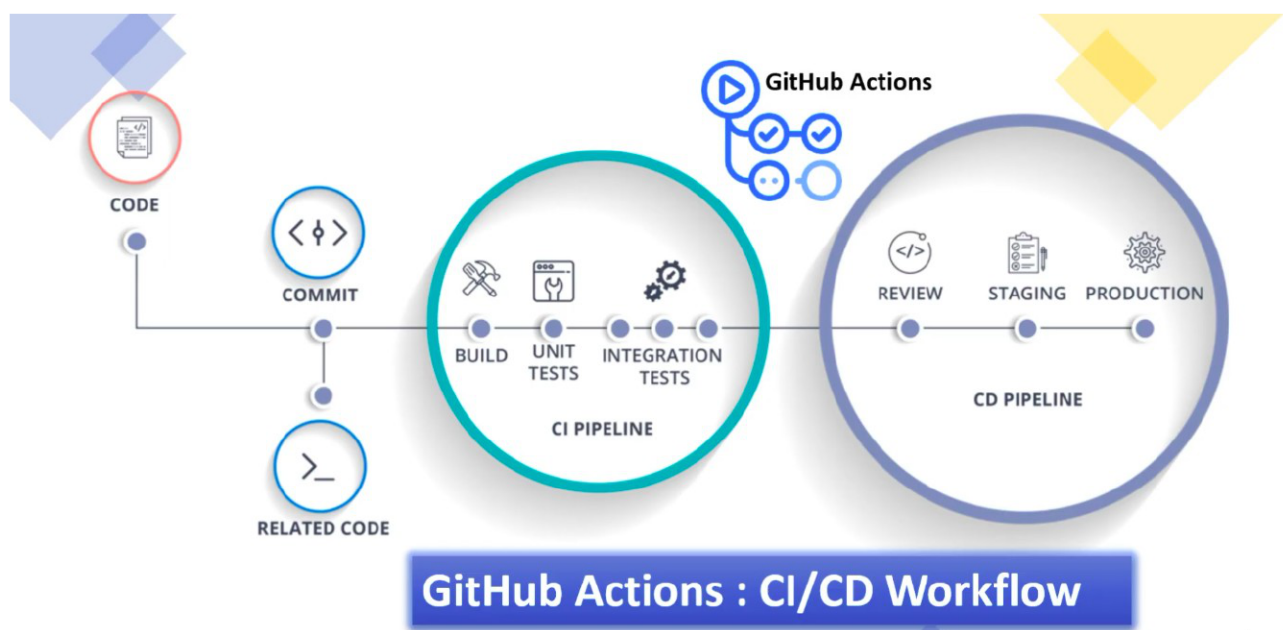


SpringBoot - Build CI/CD Pipeline Using GitHub Actions

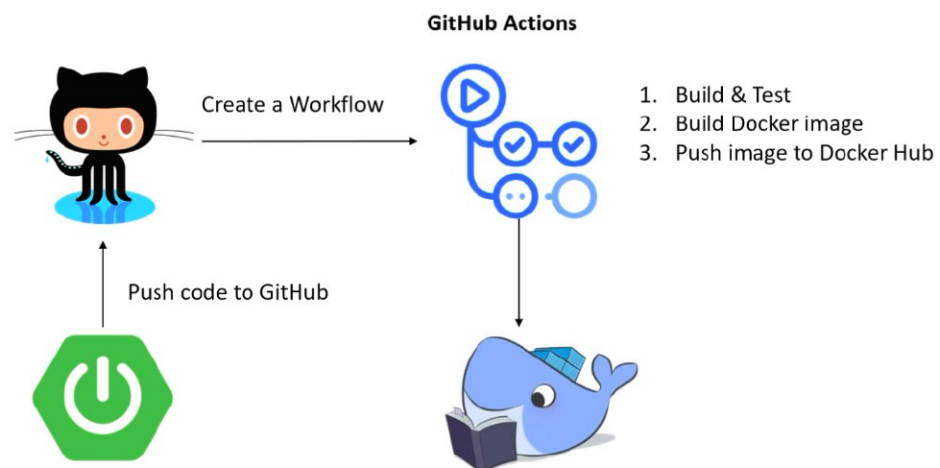
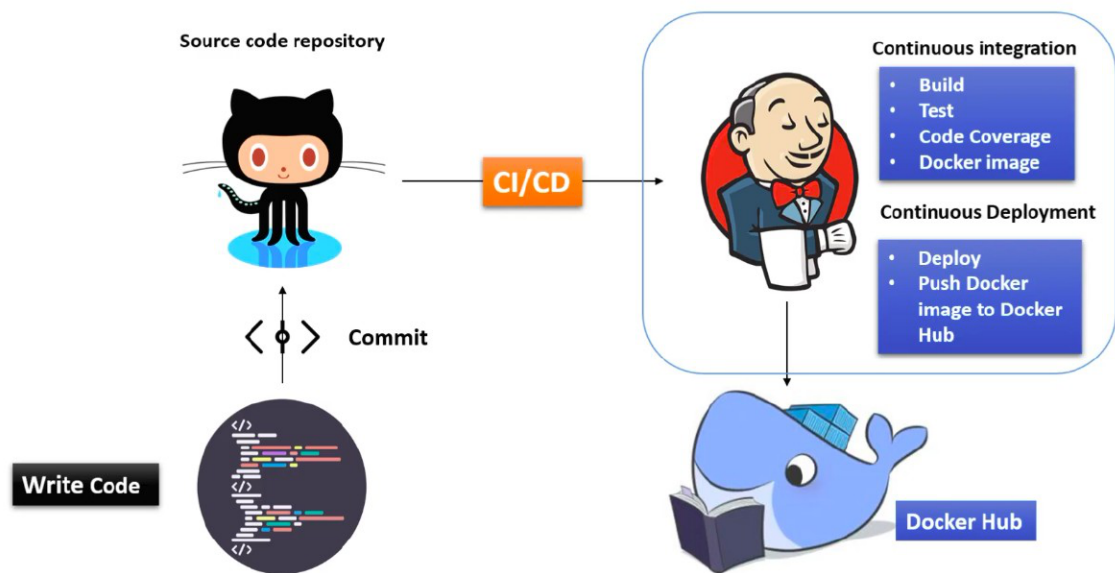
What is CI/CD?

- **CI (Continuous Integration):** Automates building & testing code whenever changes are pushed to GitHub.
- **CD (Continuous Delivery/Deployment):** Automates packaging, creating Docker images, and deploying apps (to Docker Hub, Kubernetes, or Cloud).

We'll build a **Spring Boot project**, configure **GitHub Actions** for automation, and push a **Docker image** to Docker Hub.



Jenkins as CI/CD



Step 1: First create a Spring boot sample project

- Use **Spring Initializr** or **IntelliJ** to generate a simple Spring Boot application.
- Add dependencies like **Spring Web** (or others as required).
- Test locally to ensure the project runs.

Step 2: Push the spring project on Github.

Initialize Git repository:

```
git init
```

- Add files:

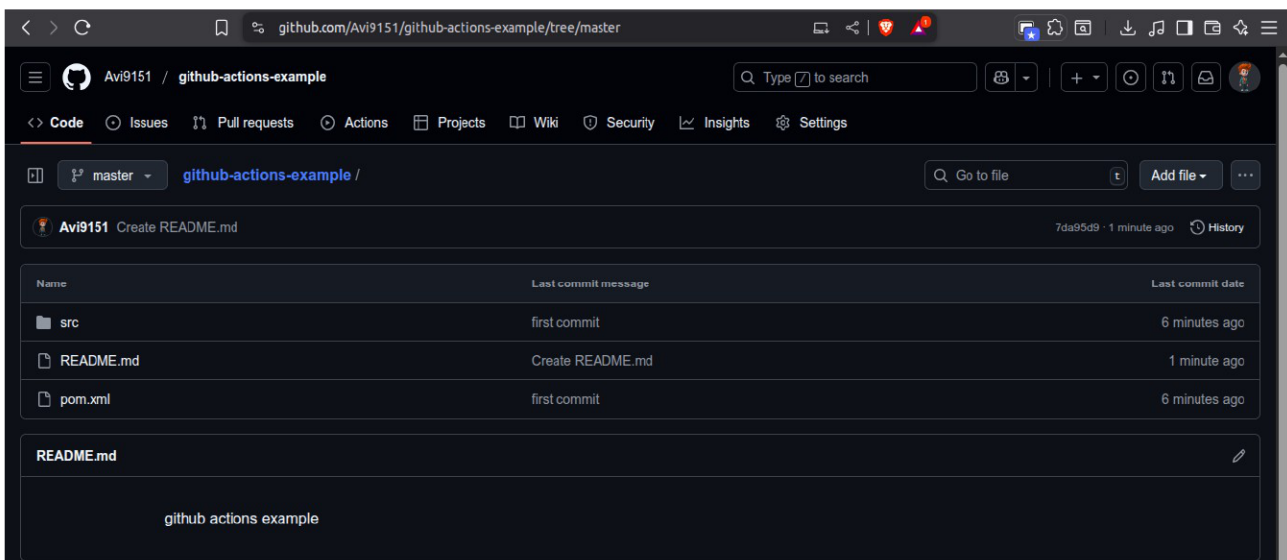
```
git add .
```

- Commit:

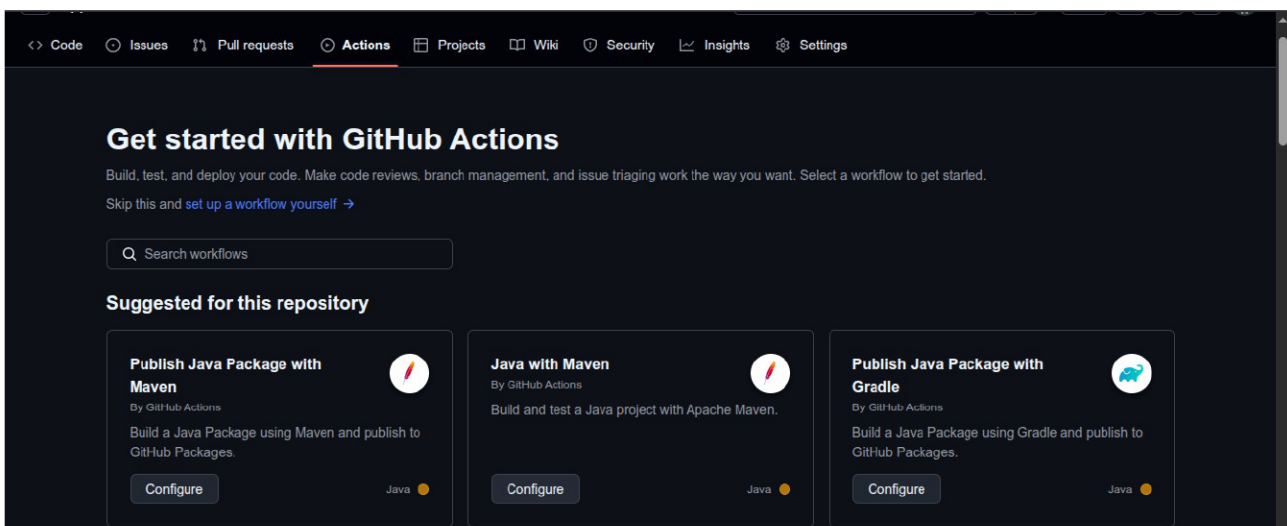
```
git commit -m "Initial commit"
```

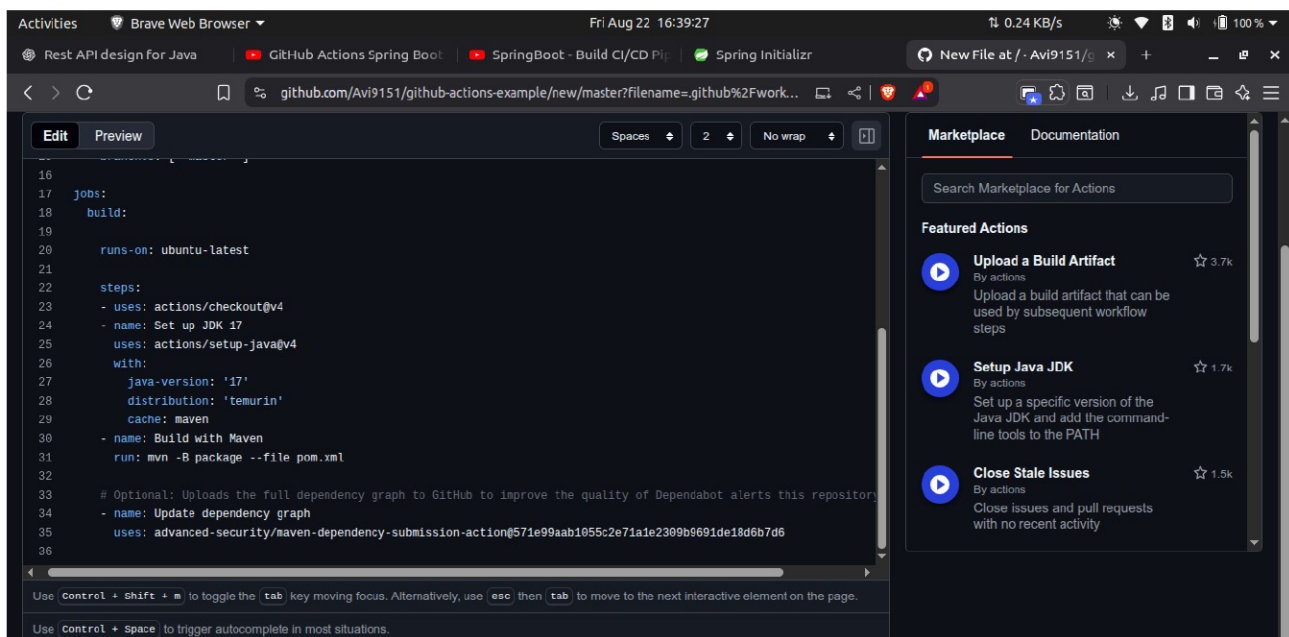
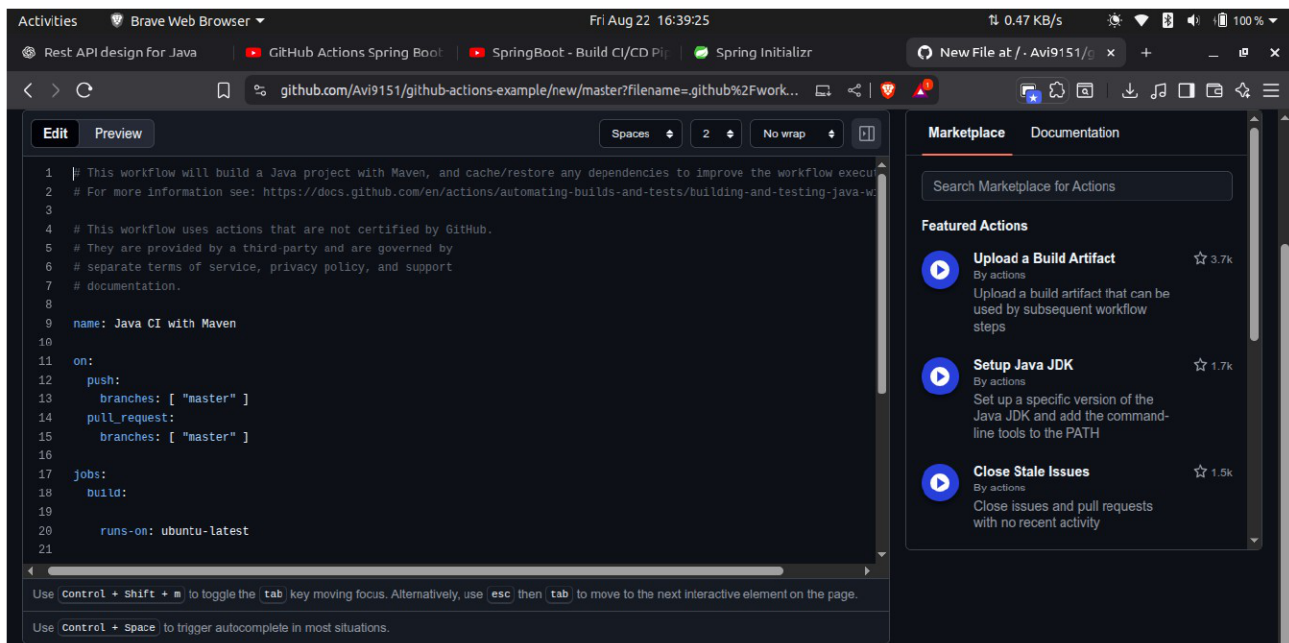
- Push to GitHub repository:

```
git remote add origin <repo-url>  
git push -u origin master
```



Step 3: Go to the **Actions** -> **Java with Maven** then click on **Configure**





Step 4: Rename the **name** and **run**

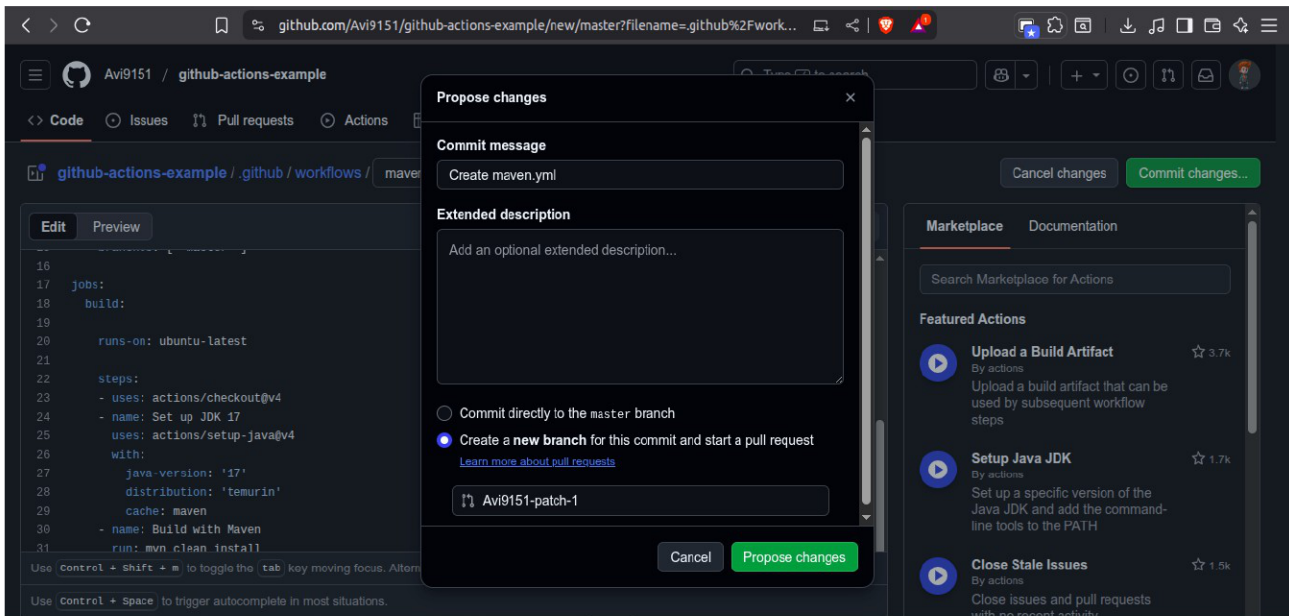
name: project Cidc flow

run: mvn clean install

Step 5: Click on the **Commit changes** button.

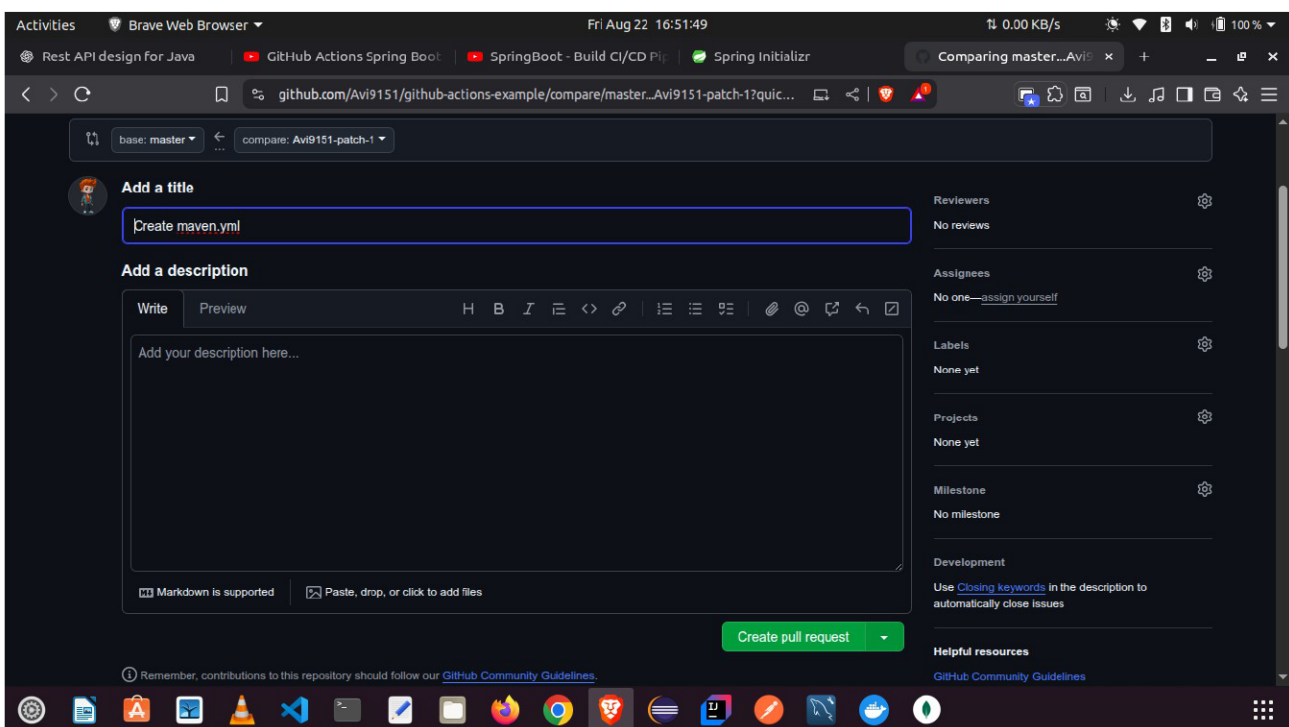
Save changes by clicking **Commit changes**.

- GitHub will suggest creating a new branch → allow it.

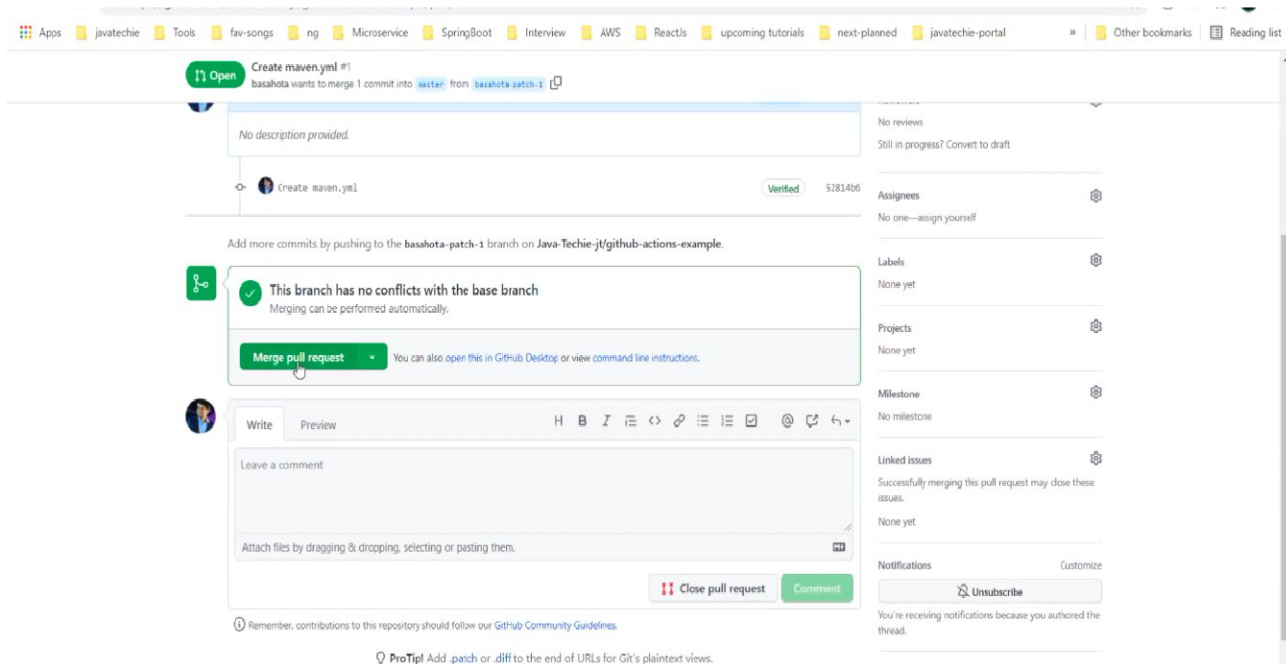


Step 6: Click on **Create a new branch for this commit and start a pull request** -> click on **Propose changes** button.

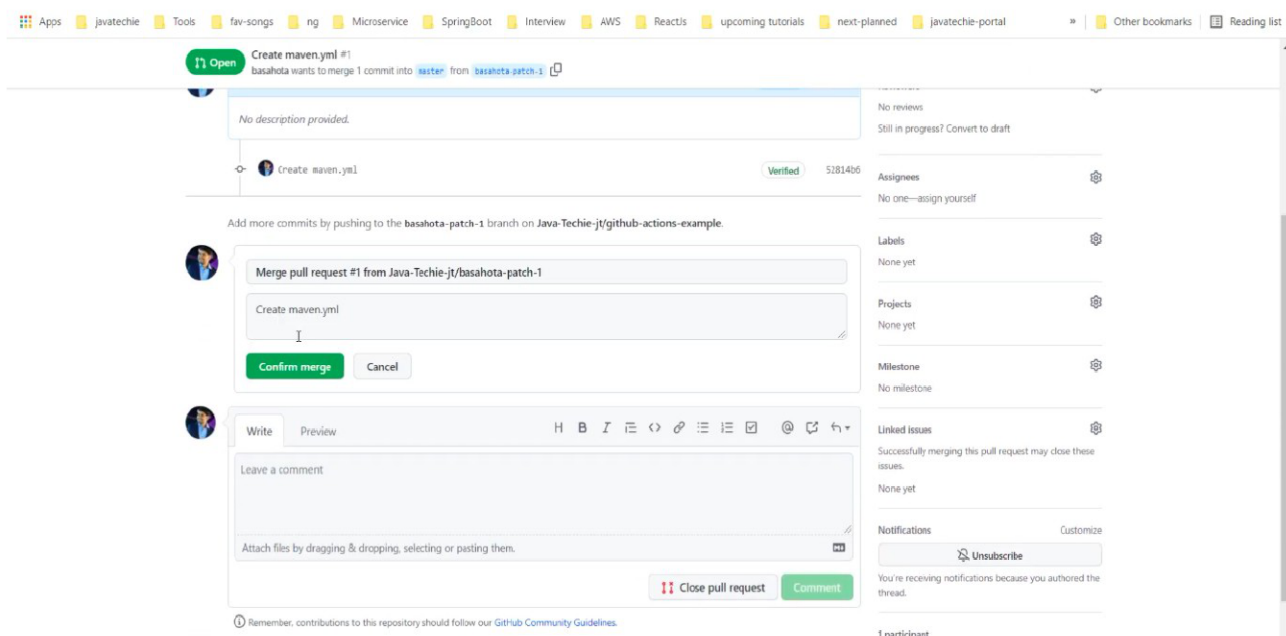
Step 7: Click on the **Create pull request** to merge these changes.



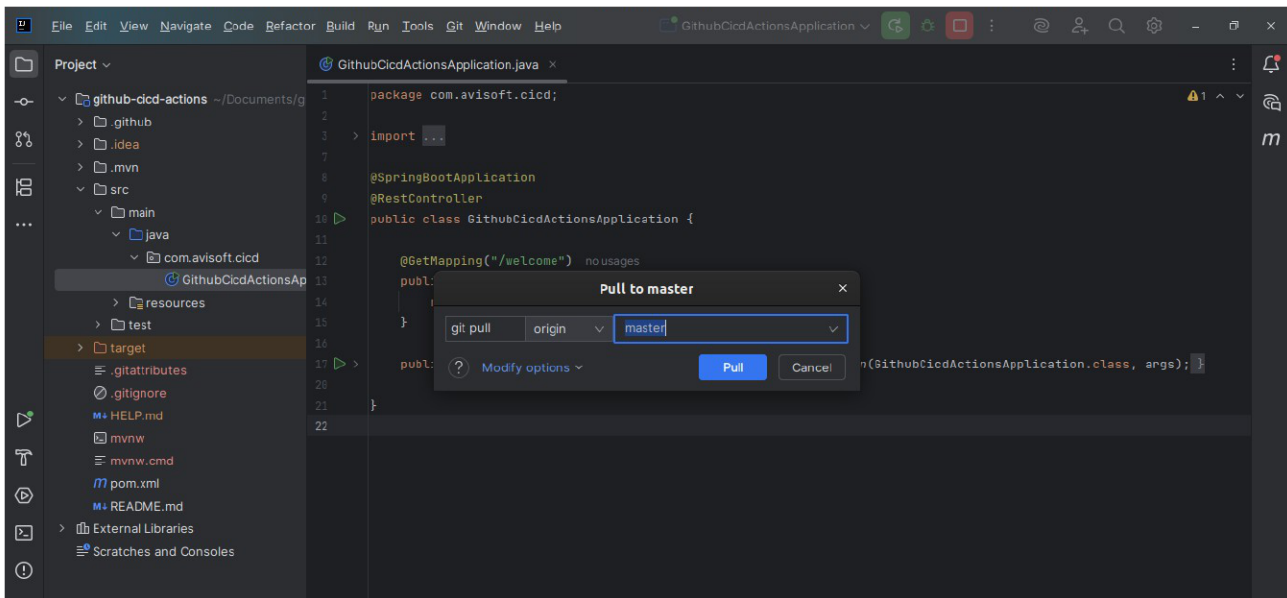
Step 8: Click on **Merge pull request**



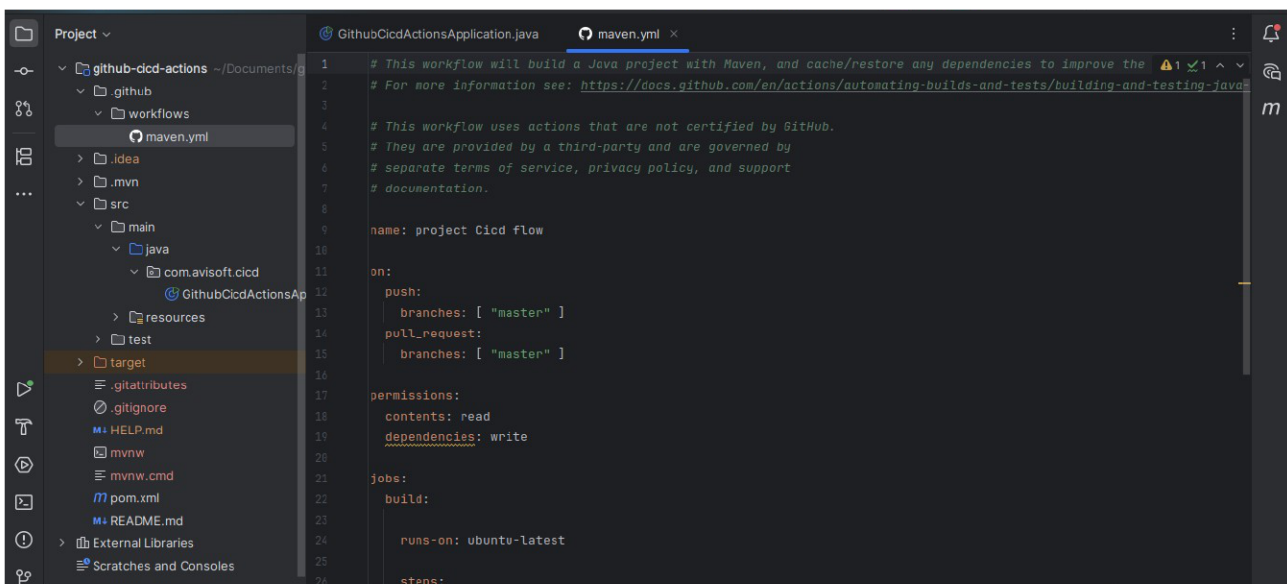
Then Click on the **Confirm merge** button.



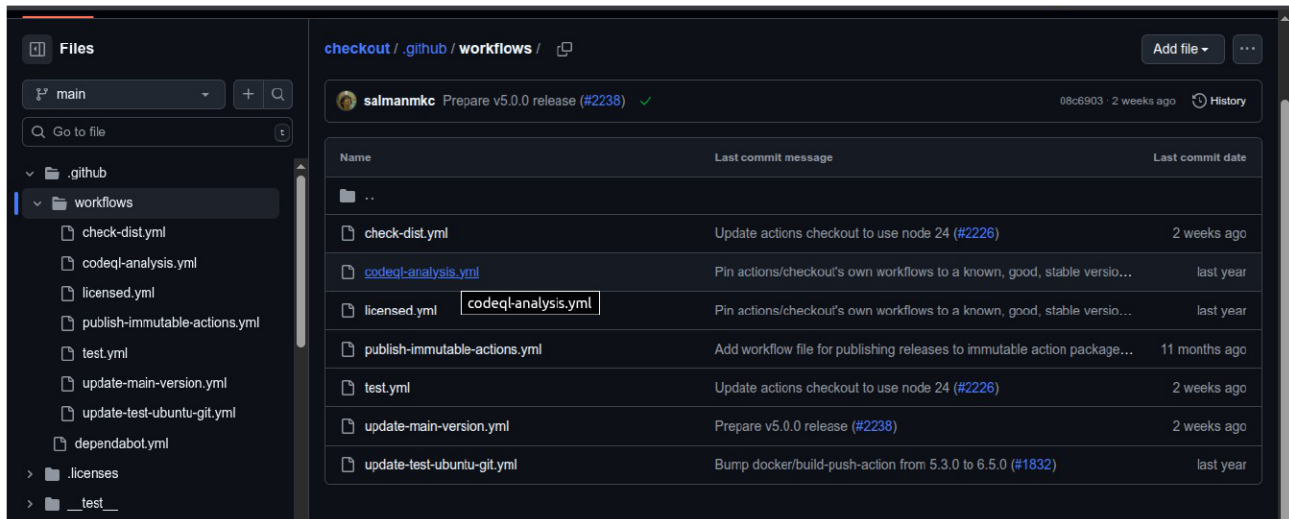
Step 9: Then Open **IntelliJ** → **git pull** → You'll see **.github/workflows/maven.yml** file added.



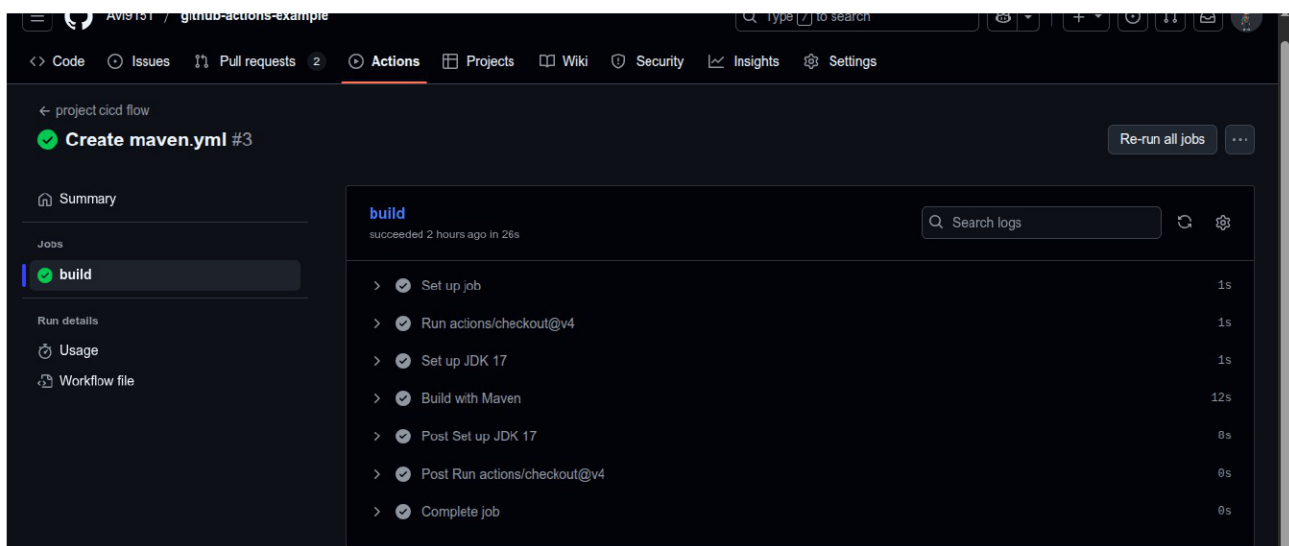
After pull open the **Github folder** -> **workflows** -> **maven.yml**



Step 10: Lets check the **actions/checkouts** so open url- <https://github.com/actions> and search.



Step 11: Go to the **Actions** and click on the Create **maven.yml**



Activities Brave Web Browser Fri Aug 22 21:29:25 0.00 KB/s 100%

GitHub Actions Spring SpringBoot - Build CI/ Spring Initializr Rest API design for J Workflow runs · A (6) YouTube

github.com/Avi9151/github-actions-example/actions

Avi9151 / github-actions-example

Code Issues Pull requests 2 Actions Projects Wiki Security Insights Settings

Workflow run deleted successfully.

Actions

New workflow

All workflows

project ci/cd flow

Management

- Caches
- Attestations
- Runners
- Usage metrics
- Performance metrics

All workflows

Showing runs from all workflows

Filter workflow runs

2 workflow runs

	Event	Status	Branch	Actor
✓ Merge pull request #3 from Avi9151/Avi9151-patch-3	project ci/cd flow #4: Commit f40e66 pushed by Avi9151	master	2 hours ago	...
✓ Create maven.yml	project ci/cd flow #3: Pull request #3 opened by Avi9151	Avi9151-patch-3	2 hours ago	...

Avi9151 / github-actions-example

Code Issues Pull requests 2 Actions Projects Wiki Security Insights Settings

← project ci/cd flow

✓ Create maven.yml #3

Re-run all jobs ...

Summary

Jobs

- build

Run details

- Usage
- Workflow file

Triggered via pull request 2 hours ago

Status	Total duration	Artifacts
Success	30s	-

maven.yml

on: pull_request

build 26s

Click on **build**

Activities Brave Web Browser Fri Aug 22 21:35:21 0.00 KB/s 100 %

GitHub Actions Spring SpringBoot - Build CI/ Spring Initializr Rest API design for Java Create maven.yml x (6) YouTube

github.com/Avi9151/github-actions-example/actions/runs/17156303819/job/48673867884

Avi9151 / github-actions-example Type to search

Code Issues Pull requests 2 Actions Projects Wiki Security Insights Settings

← project ci/cd flow

✓ Create maven.yml #3 Re-run all jobs ...

Summary

Jobs

✓ build

Run details

Usage

Workflow file

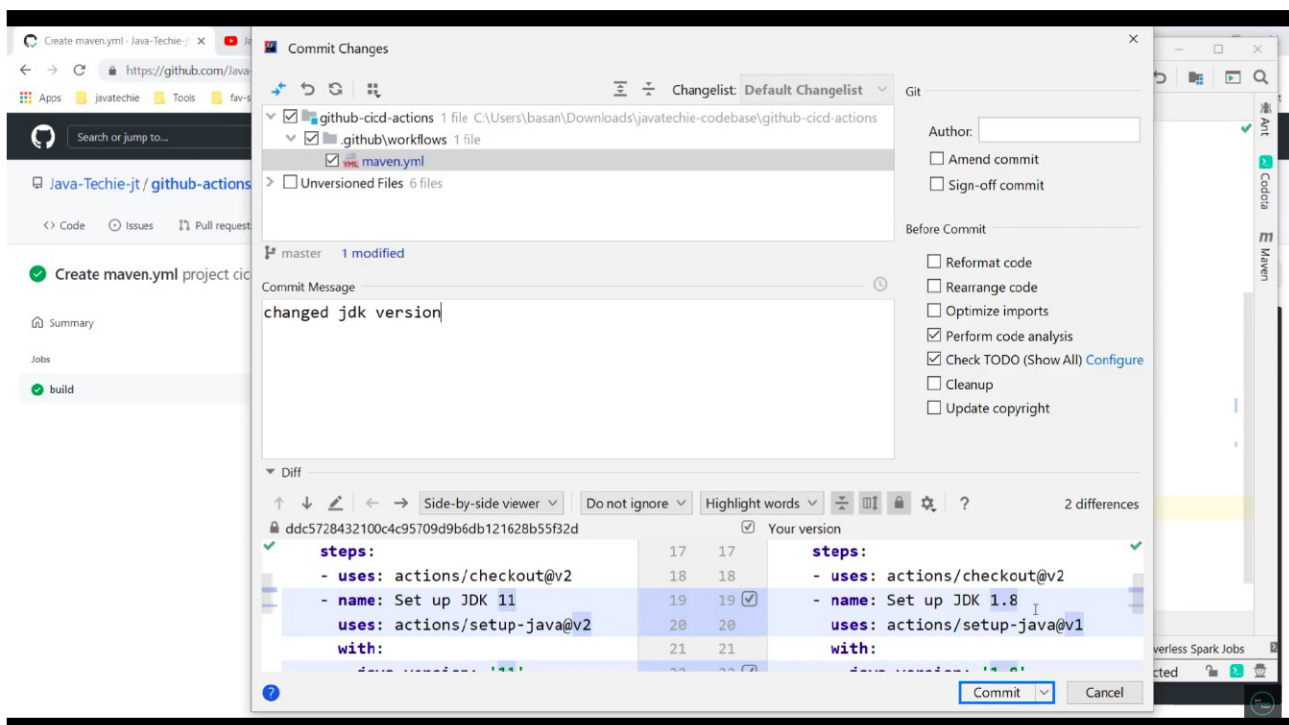
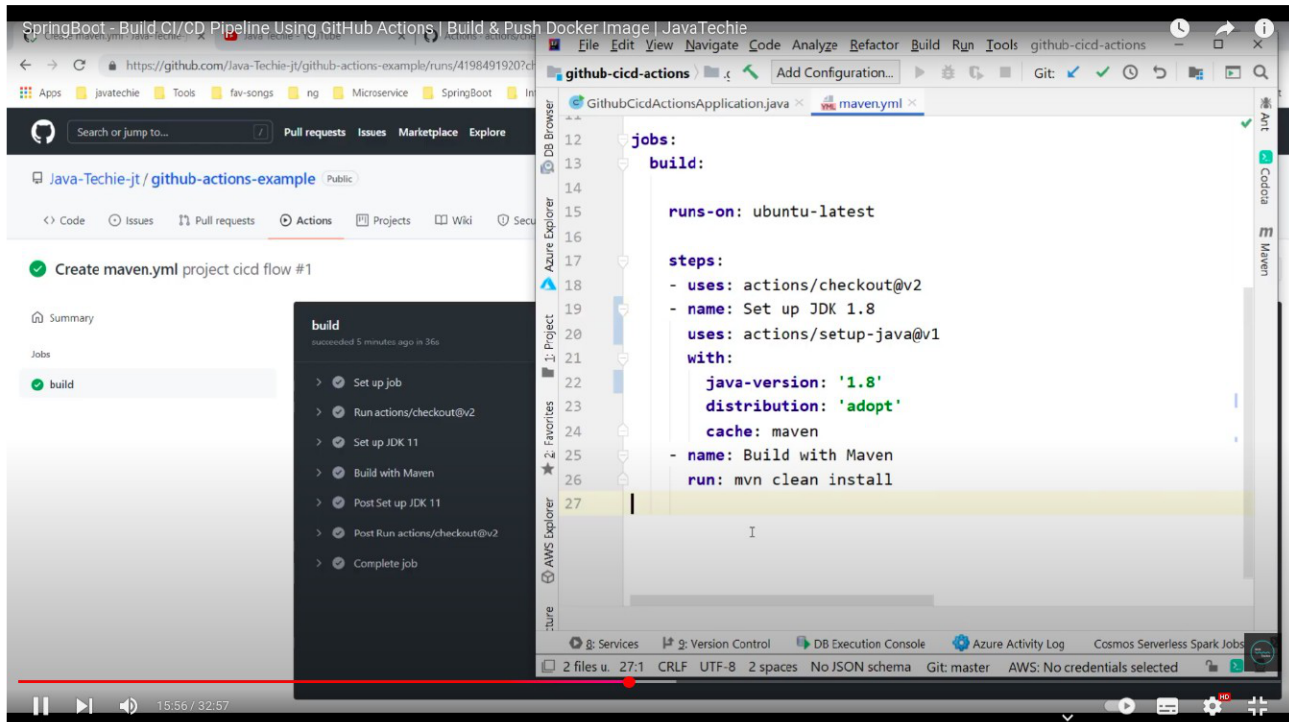
build

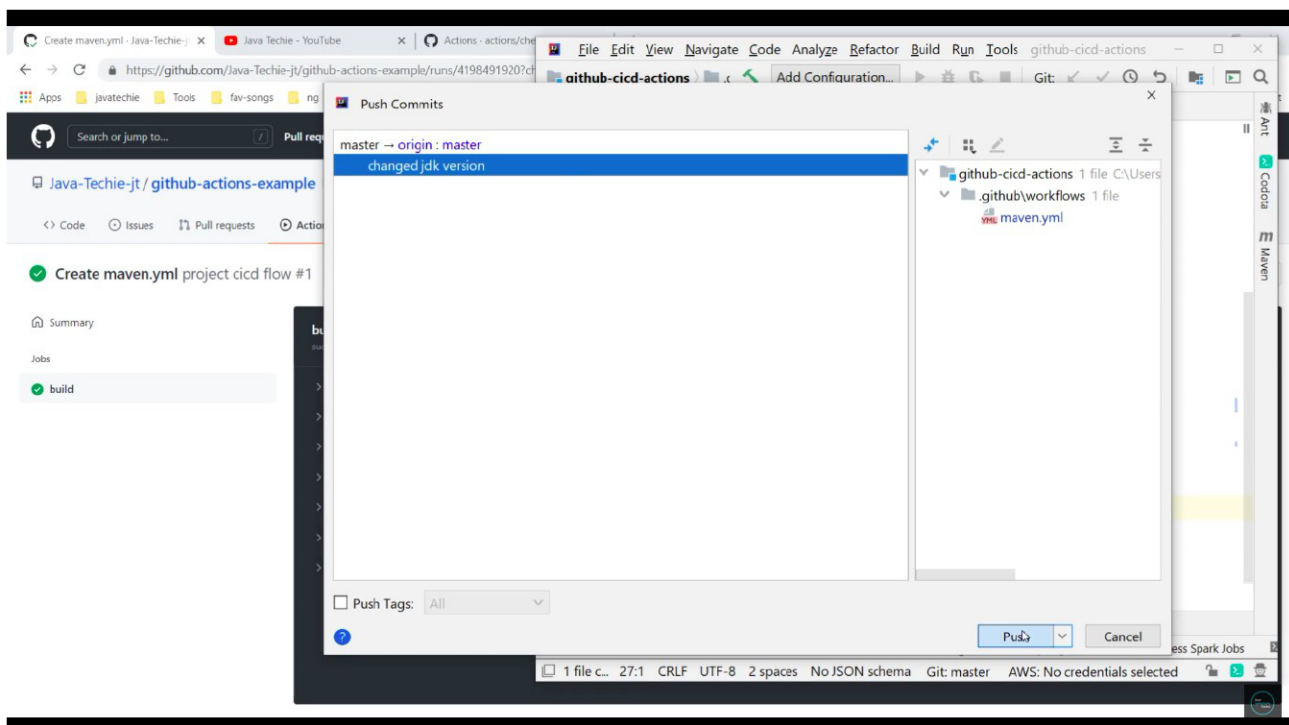
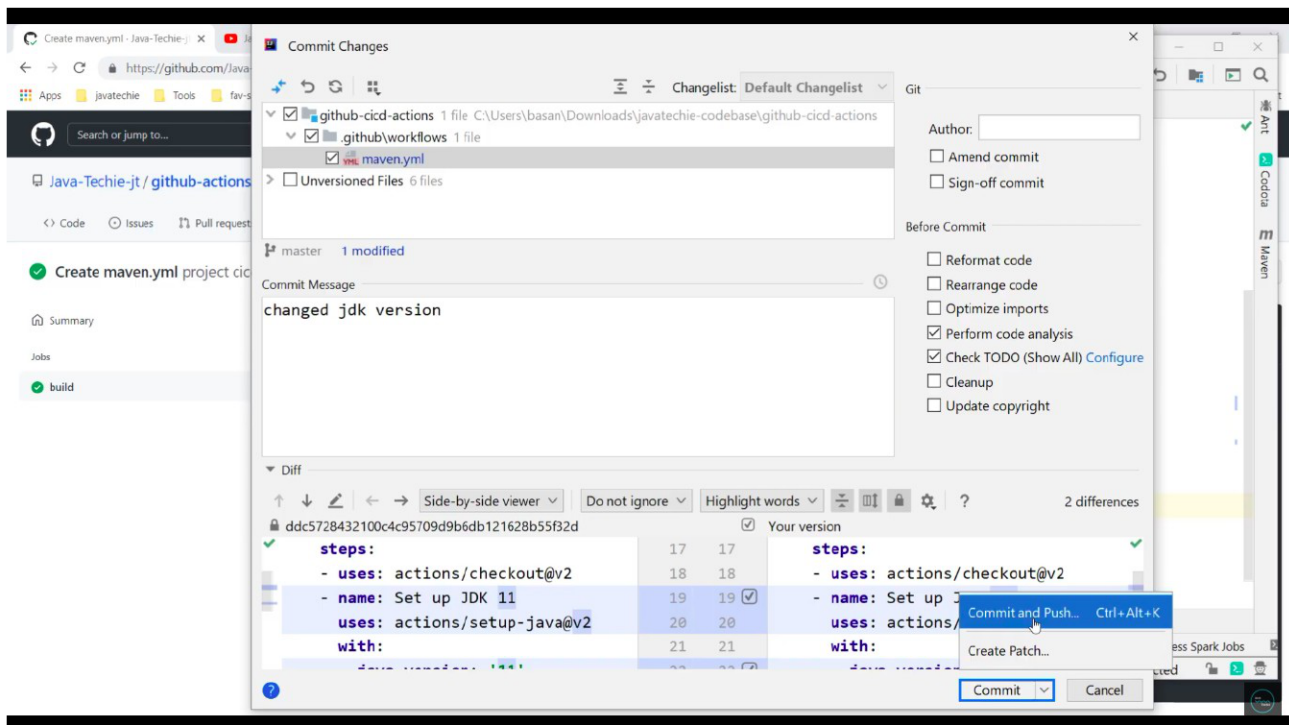
succeeded 2 hours ago in 26s

Search logs

> ✓ Set up job	1s
> ✓ Run actions/checkout@v4	1s
> ✓ Set up JDK 17	1s
> ✓ Build with Maven	12s
> ✓ Post Set up JDK 17	0s
> ✓ Post Run actions/checkout@v4	0s
> ✓ Complete job	0s

Note : If you want to change **jdk** version and [checkout@v](#)(version).





Click on **changed jdk version**

The screenshot shows the GitHub Actions interface for the repository 'Java-Techie-jt / github-actions-example'. The 'All workflows' tab is selected, displaying a list of workflow runs. The first run, titled 'changed jdk version', is highlighted. It shows the workflow was triggered by a push to the 'master' branch. The status is 'Queued'. Below the list, there is a section for 'All workflows' showing runs from all workflows. The 'changed jdk version' workflow is the first one listed.

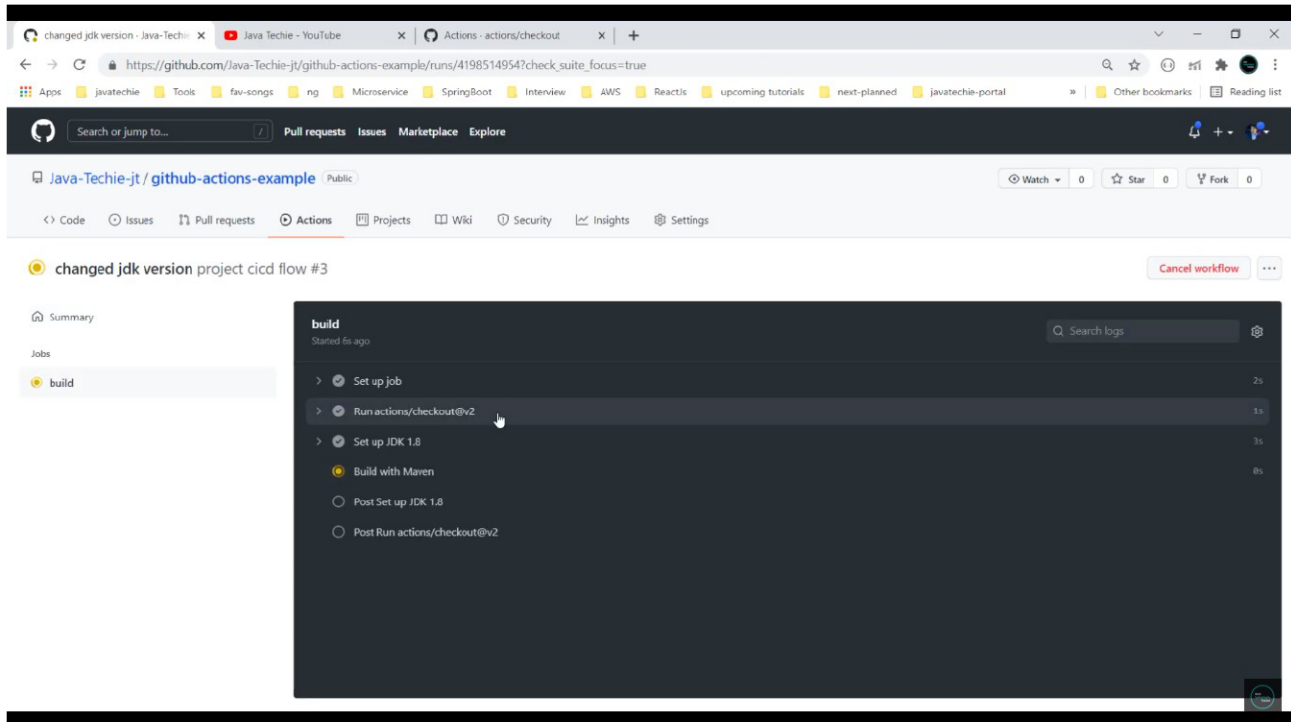
Workflow runs	Event	Status	Branch	Actor
changed jdk version project cid flow #3: Commit 14d0b7b pushed by basahota	push	Queued	master	basahota
Merge pull request #1 from Java-Techie-jt/basahota-patch-1 project cid flow #2: Commit odc5728 pushed by basahota	pull_request	Completed	master	basahota
Create maven.yml project cid flow #1: Pull request #1 opened by basahota	pull_request	Completed	basahota-patch-1	basahota

Click on **build**

The screenshot shows the GitHub Actions interface for the repository 'Java-Techie-jt / github-actions-example'. The 'changed jdk version' workflow is selected, and the 'build' job is highlighted. The job is currently in progress. The workflow was triggered by a push to the 'master' branch. The status is 'In progress'. The total duration is 2s. The artifacts section shows the 'build' job.

Summary	Triggered via push 10 seconds ago	Status	Total duration	Artifacts
Summary	basahota pushed 14d0b7b master	In progress	2s	build

You can see your all **jdk version** change, etc(if you want to change) it is doing setup.

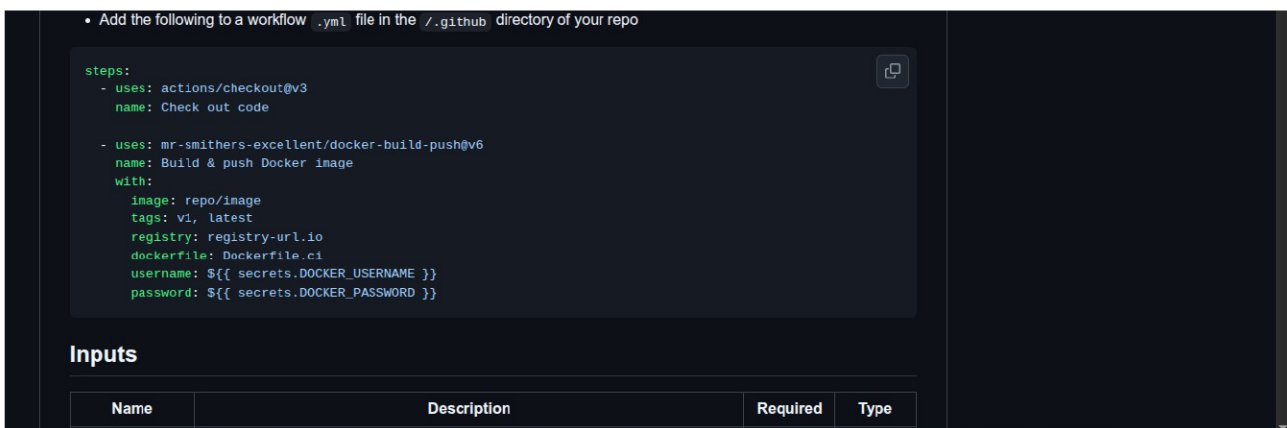
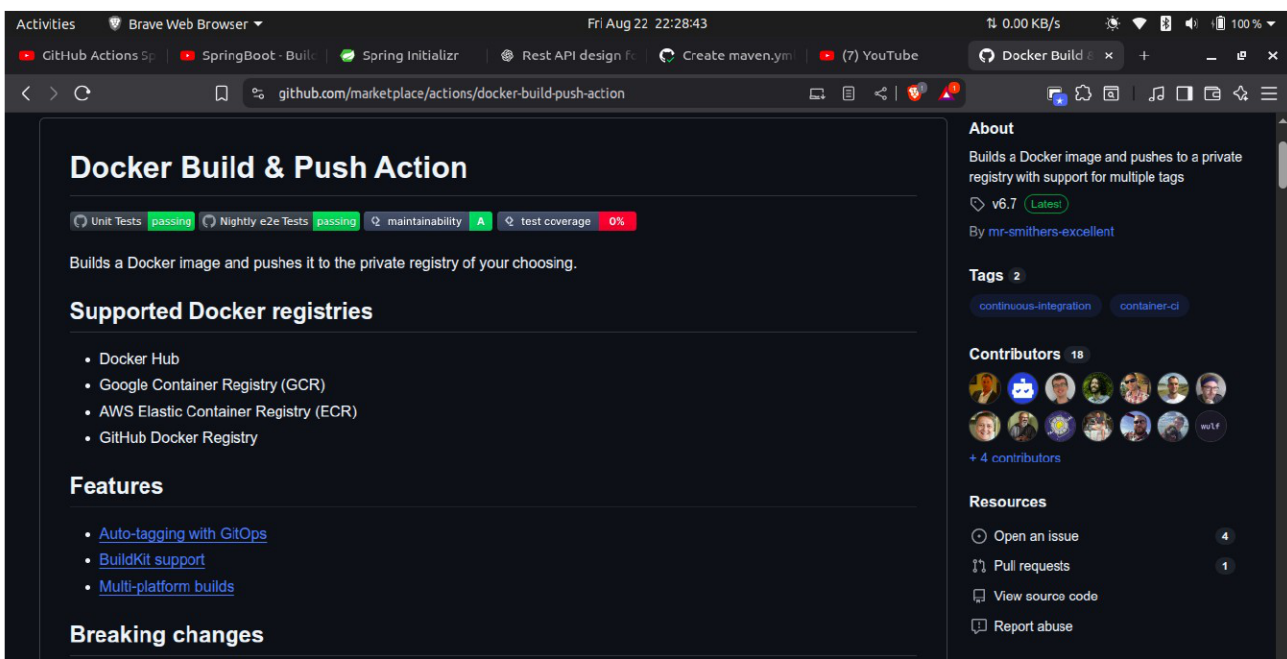
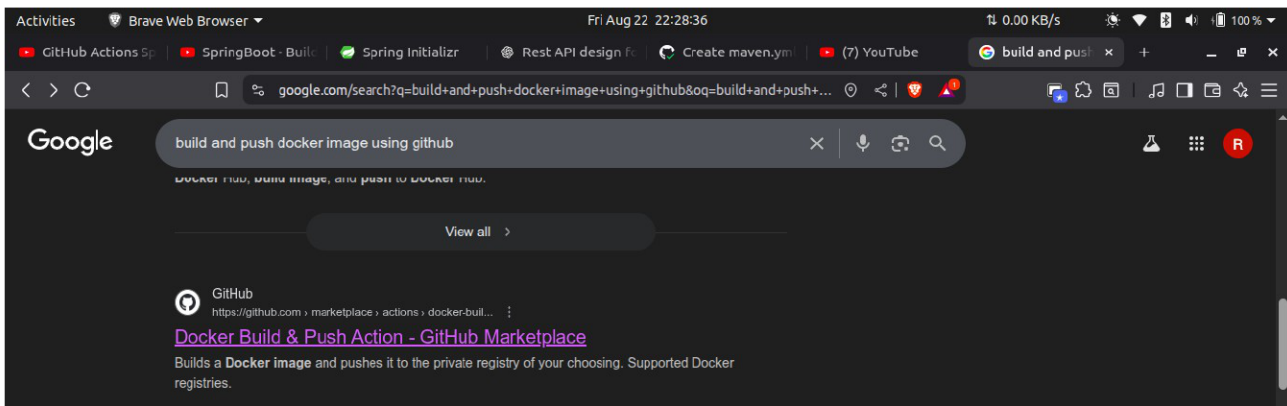


Now we tell to the Actions to build docker image

1. Login to your docker hub
2. Build a docker image
3. Push that image to docker hub

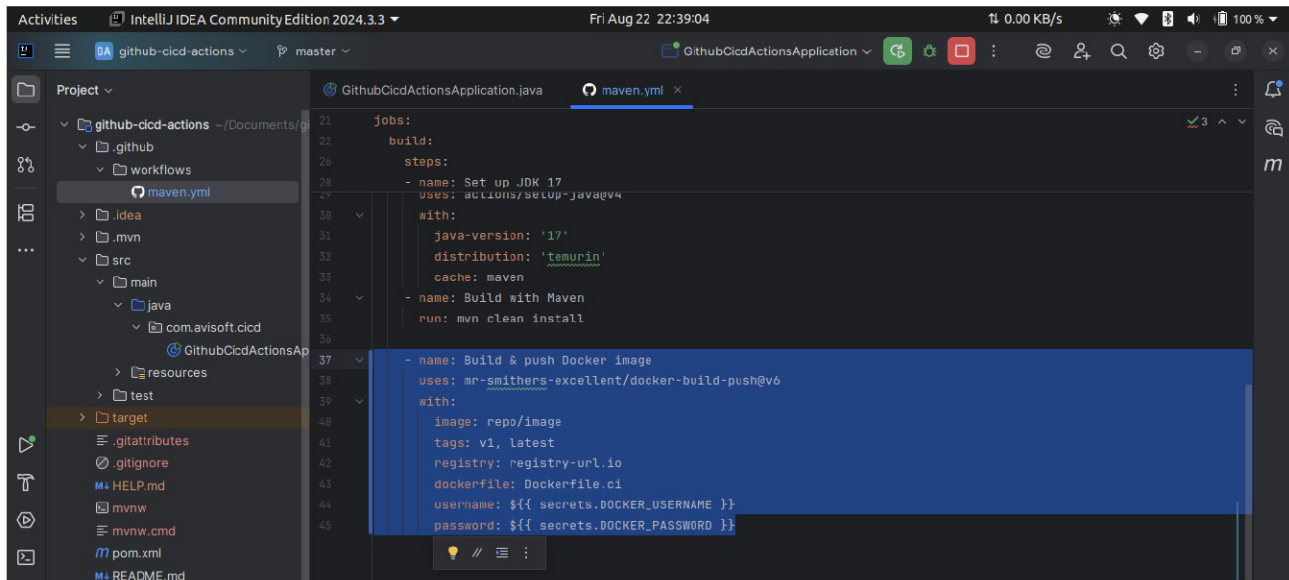
Note : “Github provide readymade Actions which can create docker image and push automatically without manually configuration.”

Step 12: Now search this - “build and push docker image using github” on browser.



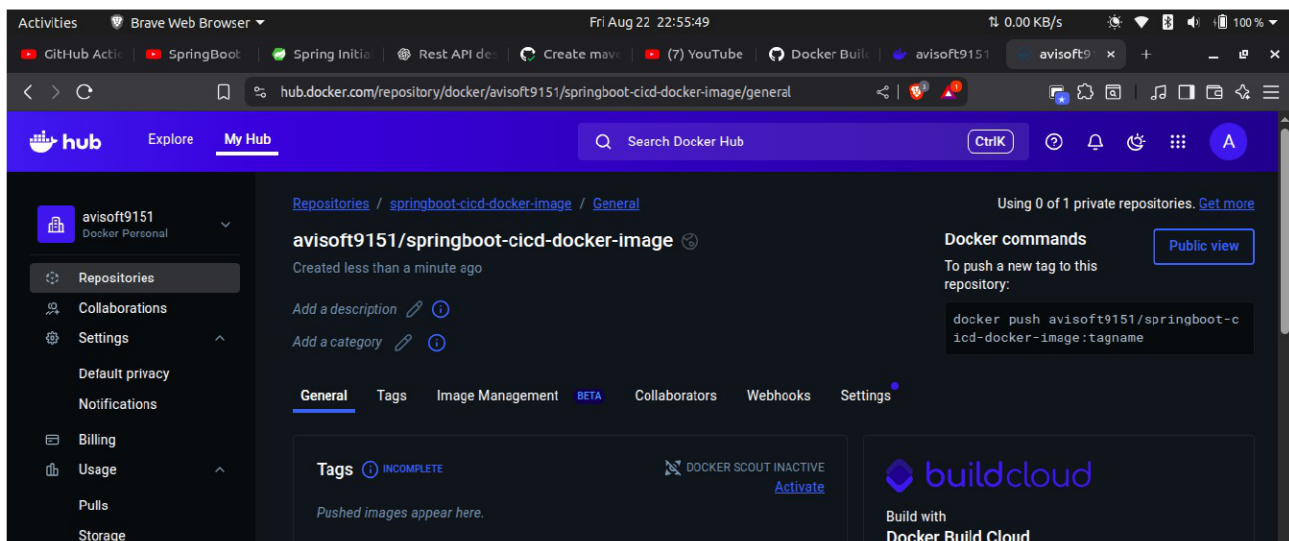
Step 13: Add this yml code in maven.yml.

```
- name: Build & push Docker image
uses: mr-smithers-excellent/docker-build-push@v6
with:
  image: repo/image
  tags: v1, latest
  registry: registry-url.io
  dockerfile: Dockerfile
  username: ${ secrets.DOCKER_USERNAME }}
  password: ${ secrets.DOCKER_PASSWORD }}
```



Step 14: Open Docker hub on browser and login.

Step 15: Create new Repository.



Step 16: Now update `maven.yml` code.

`name:` project cicd flow

`on:`

`push:`

`branches: [ "master" ]`

`pull_request:`

`branches: [ "master" ]`

`permissions:`

`contents: write`

`security-events: write`

`jobs:`

`build:`

`runs-on: ubuntu-latest`

`steps:`

`- uses: actions/checkout@v4`

`- name: Set up JDK 17`

`uses: actions/setup-java@v4`

`with:`

`java-version: '17'`

`distribution: 'temurin'`

`cache: maven`

`- name: Build with Maven`

`run: mvn clean install`

Update from here

- name: Build & push Docker image

uses: mr-smithers-excellent/docker-build-push@v6

with:

image: avisoft9151/springboot-cicd-docker-image

tags: latest

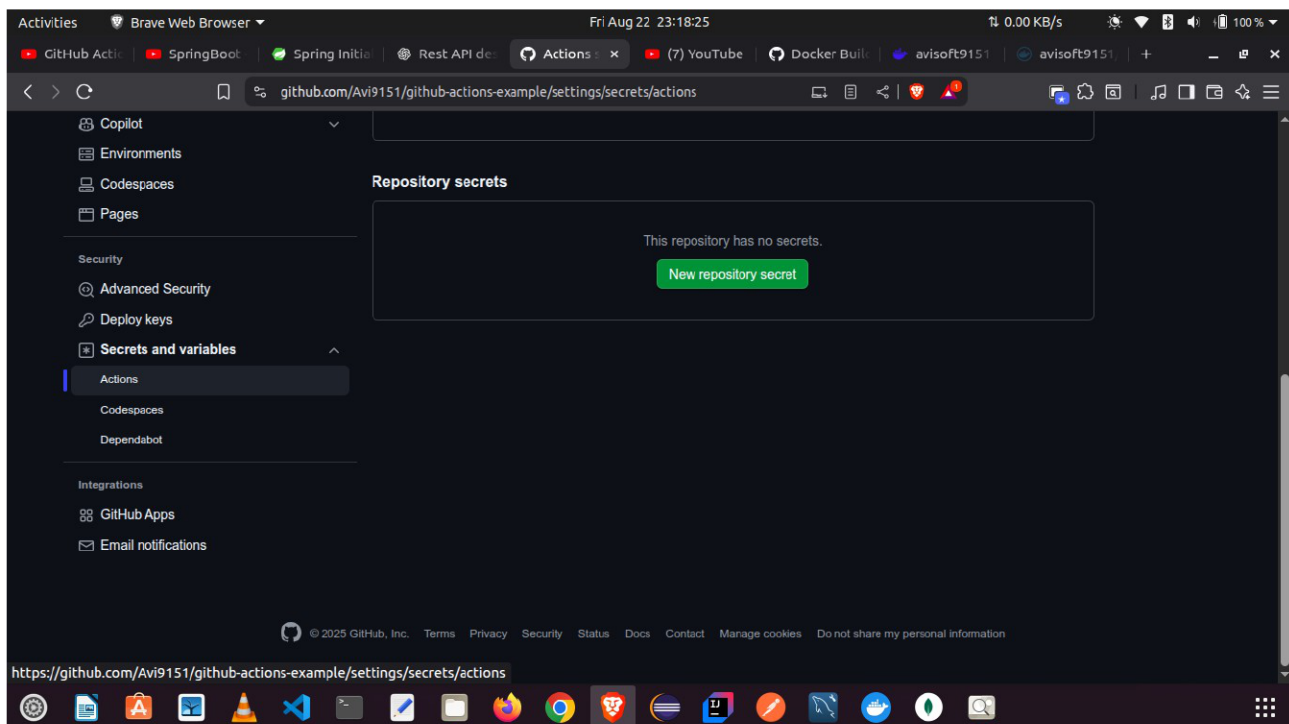
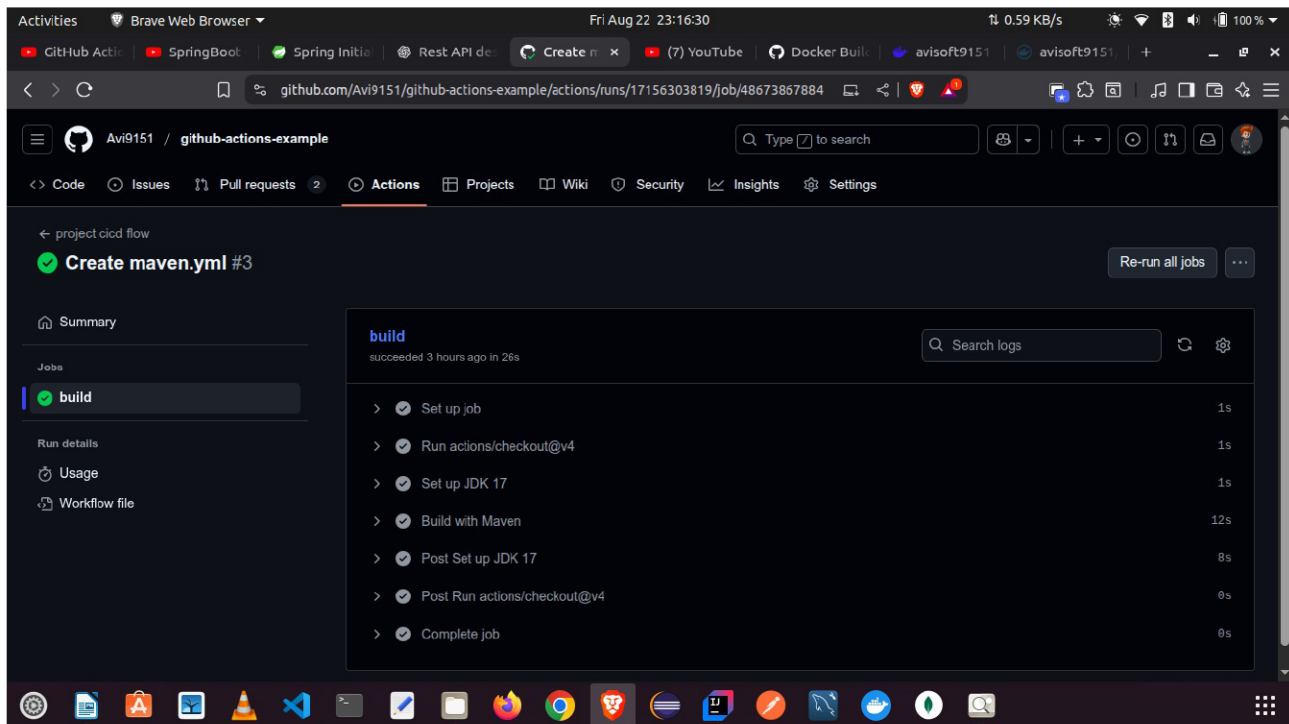
registry: docker.io

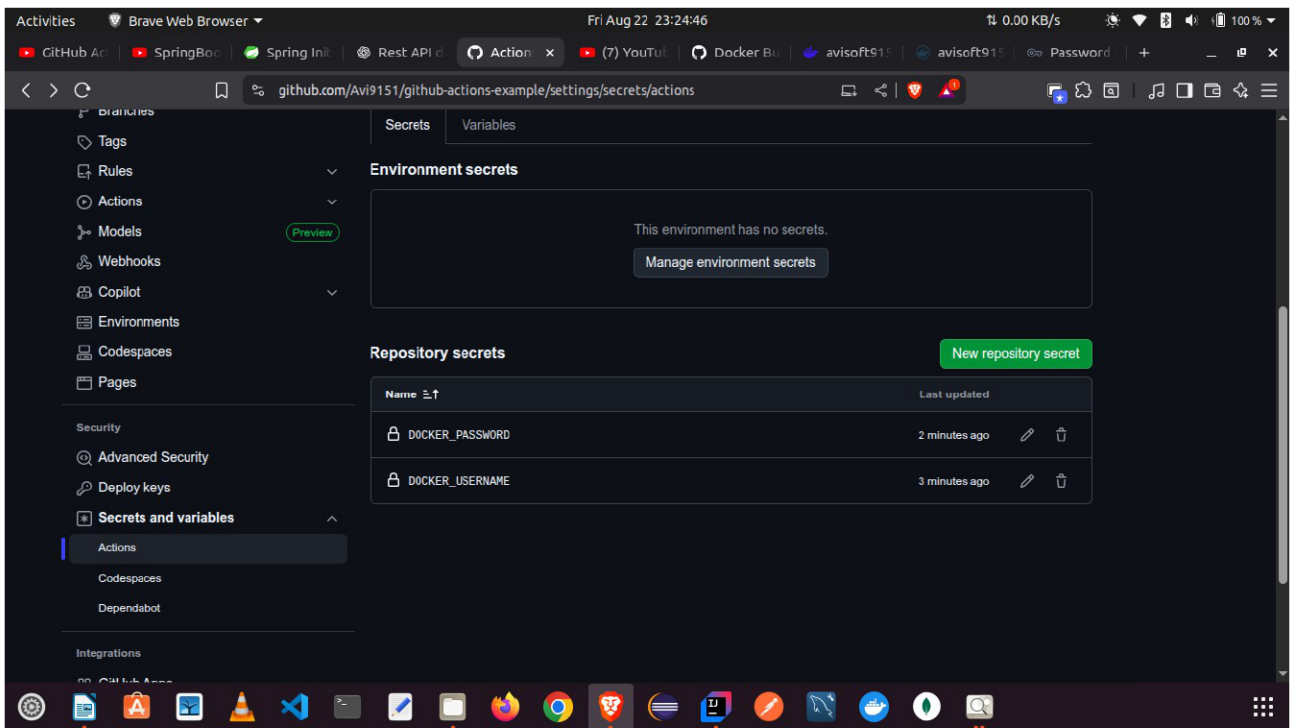
dockerfile: Dockerfile

username: \${ secrets.DOCKER_USERNAME }}

password: \${ secrets.DOCKER_PASSWORD }}

Step 17: Go to the github repo and click on **settings** -> **Secrets and Variables** -> **Actions** -> **New Repository Secrets** -> **Add Secrets** for **DOCKER_USERNAME** and **DOCKER_PASSWORD**





Step 18: Create docker file, click on [github-cicd-actions](#) -> create file [Dockerfile](#) and write code.

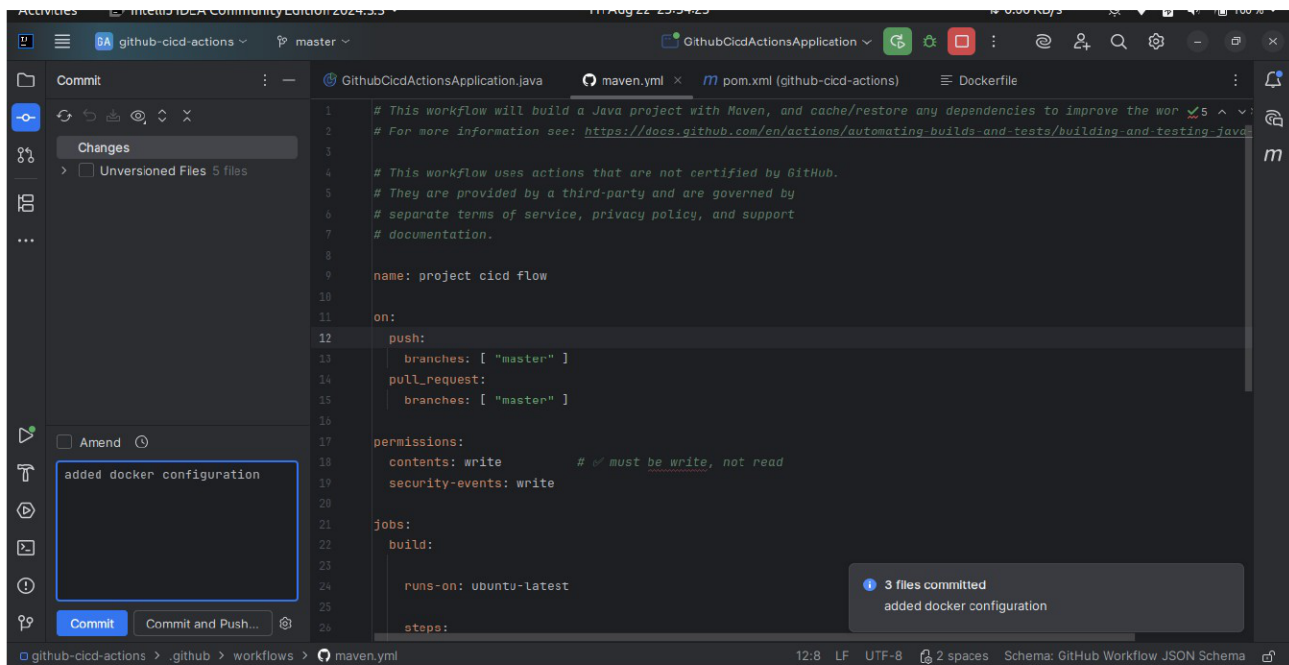
```
FROM openjdk:17
EXPOSE 8080
ADD target/springboot-cicd-docker-image.jar springboot-cicd-docker-image.jar
ENTRYPOINT ["java", "-jar", "/springboot-cicd-docker-image.jar"]
```

Step 19: Update [pom.xml](#) code and add this [finalName](#).

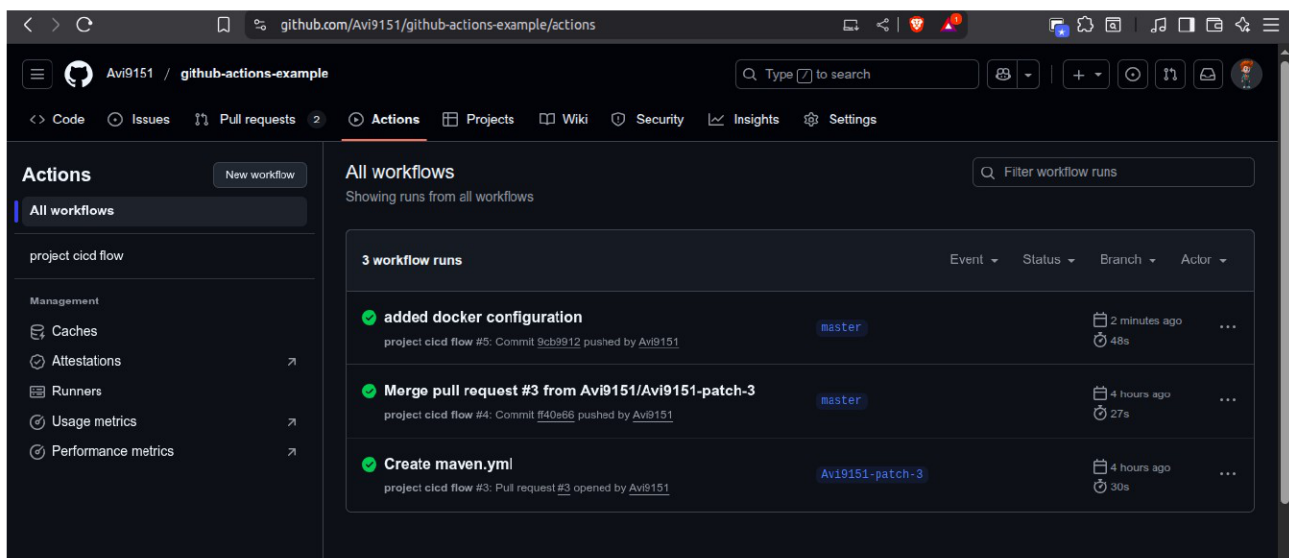
```
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
    </plugin>
  </plugins>
  <finalName>springboot-cicd-docker-image</finalName>
</build>
```

Note : It will create docker image and push to the docker hub.

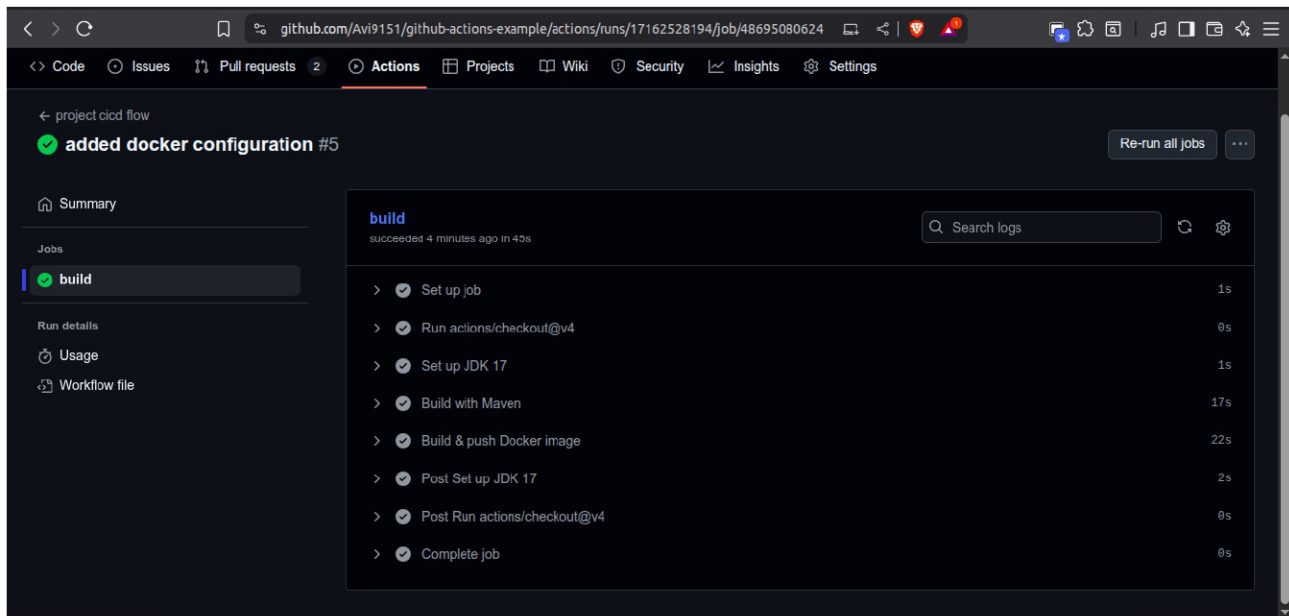
Step 20: Now open IntelliJ Idea and git commit and push.



Step 21: Now go to the [github](#) and click on [Actions](#) and you can see started the build.

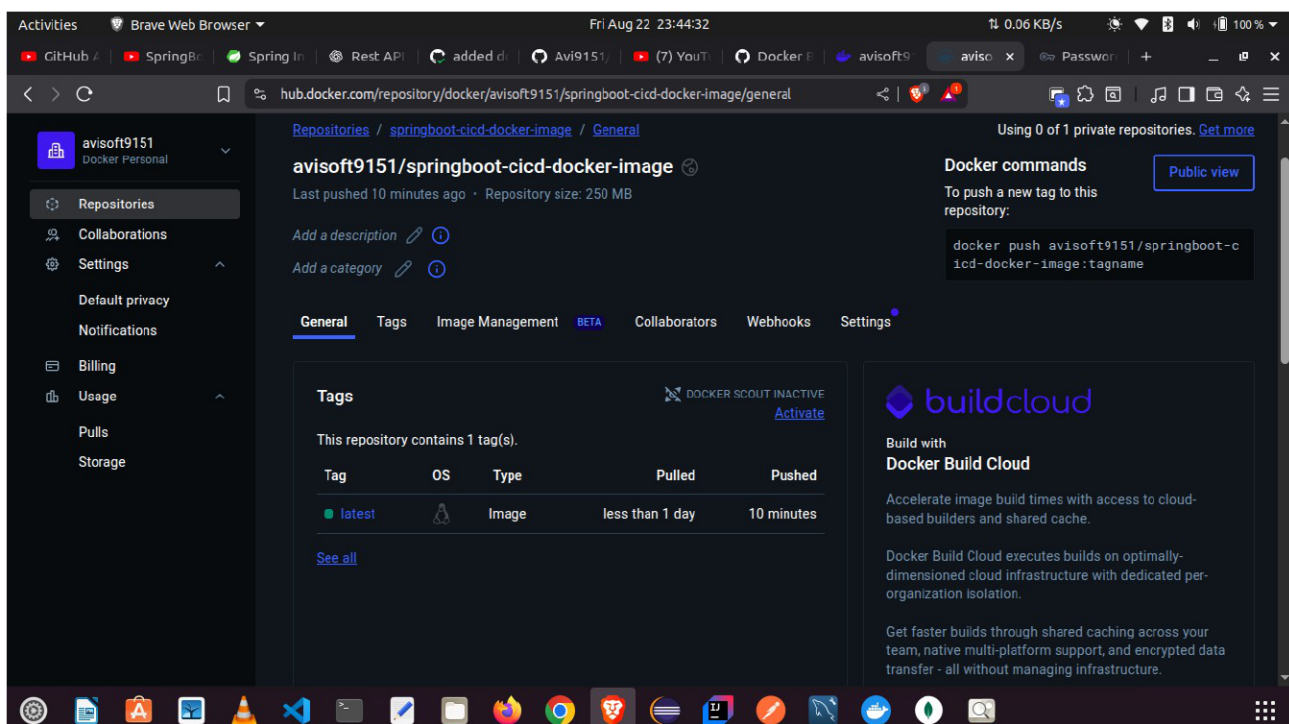


Step 22: Click on the **added docker configuration** -> **build** and you can see start build with maven



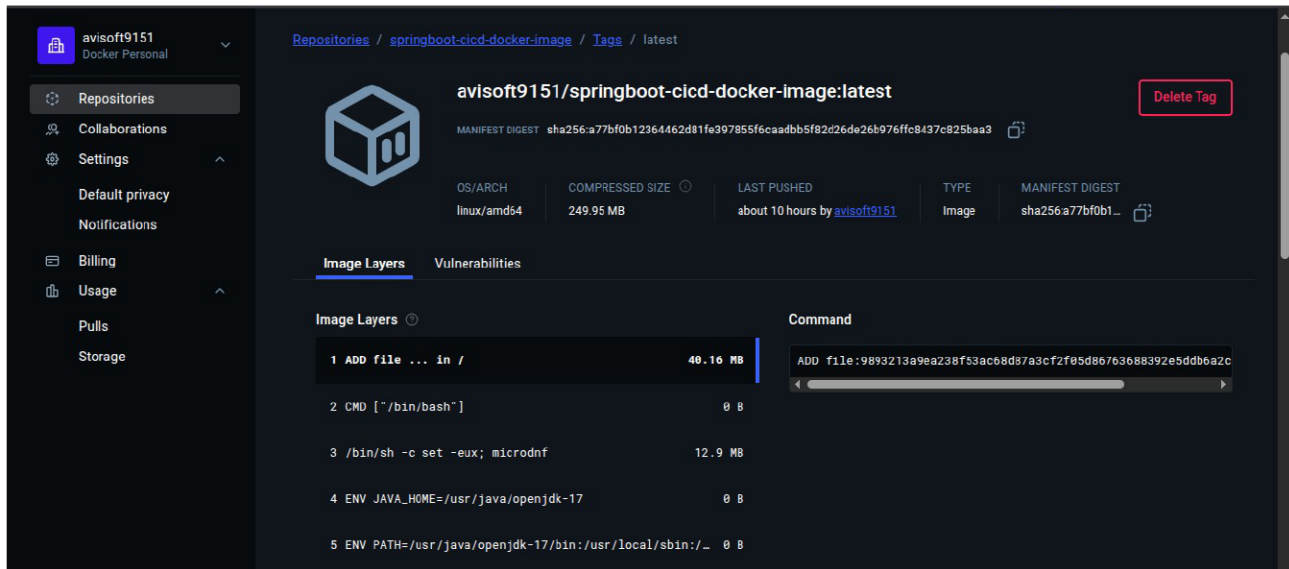
Note : Now you can see all build process completed and it should push the image to docker hub

Step 23: Now open the Docker hub on browser and refresh the page.



Note : Now you can see image got pushed on the docker hub.

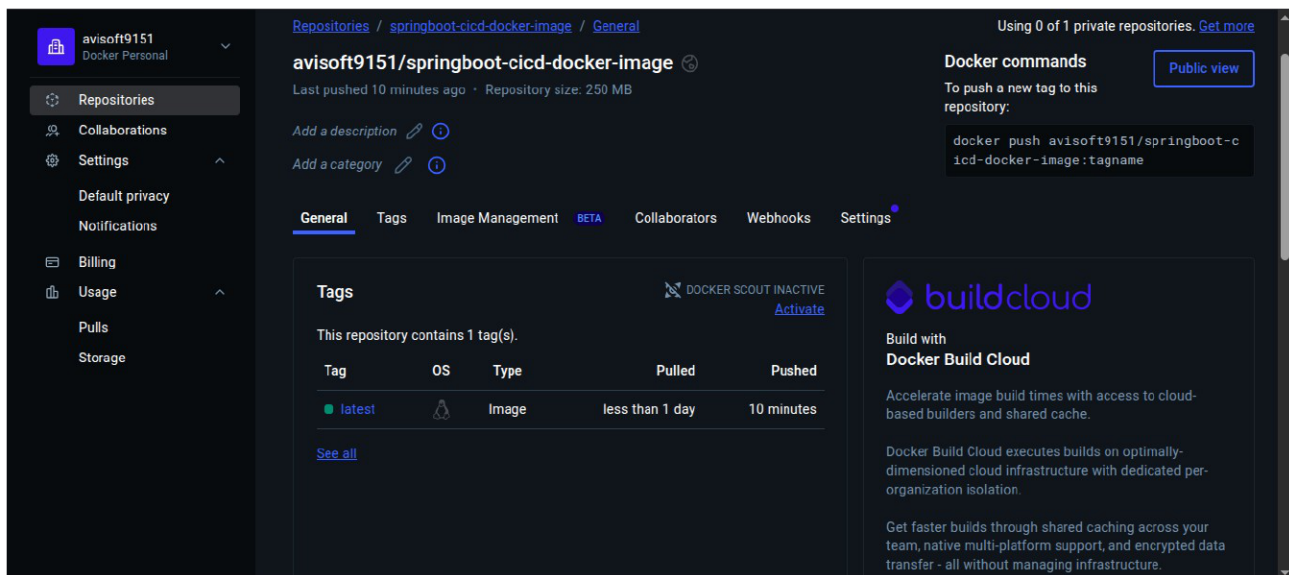
And you can click on [latest](#) tag and see.



The screenshot shows the Docker Hub interface for the repository `avisoft9151/springboot-cicd-docker-image` with the `latest` tag selected. The left sidebar contains navigation links: Repositories, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area displays the repository name, a 'Delete Tag' button, and a table of image layers. The 'Image Layers' tab is active, showing a list of layers with their commands and sizes. The 'Command' tab is also visible, showing the command used to build the image.

Layer	Command	Size
1	ADD file ... in /	40.16 MB
2	CMD ["/bin/bash"]	0 B
3	/bin/sh -c set -eux; microdnf	12.9 MB
4	ENV JAVA_HOME=/usr/java/openjdk-17	0 B
5	ENV PATH=/usr/java/openjdk-17/bin:/usr/local/sbin:/...	0 B

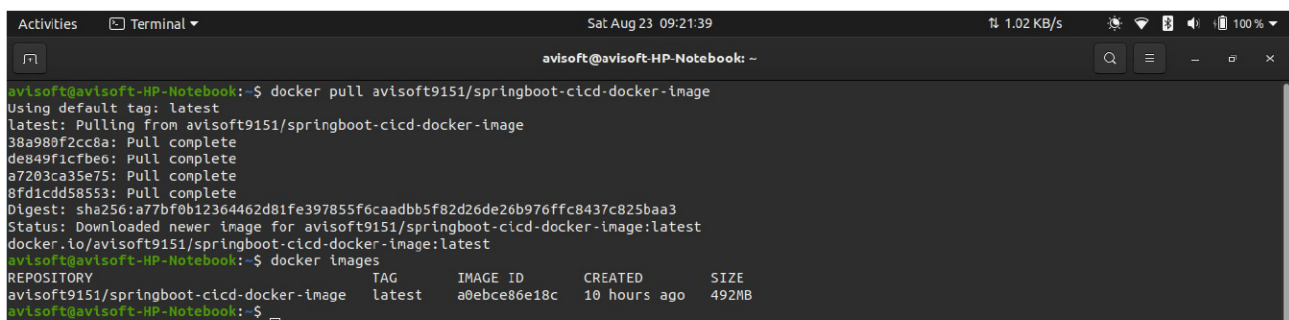
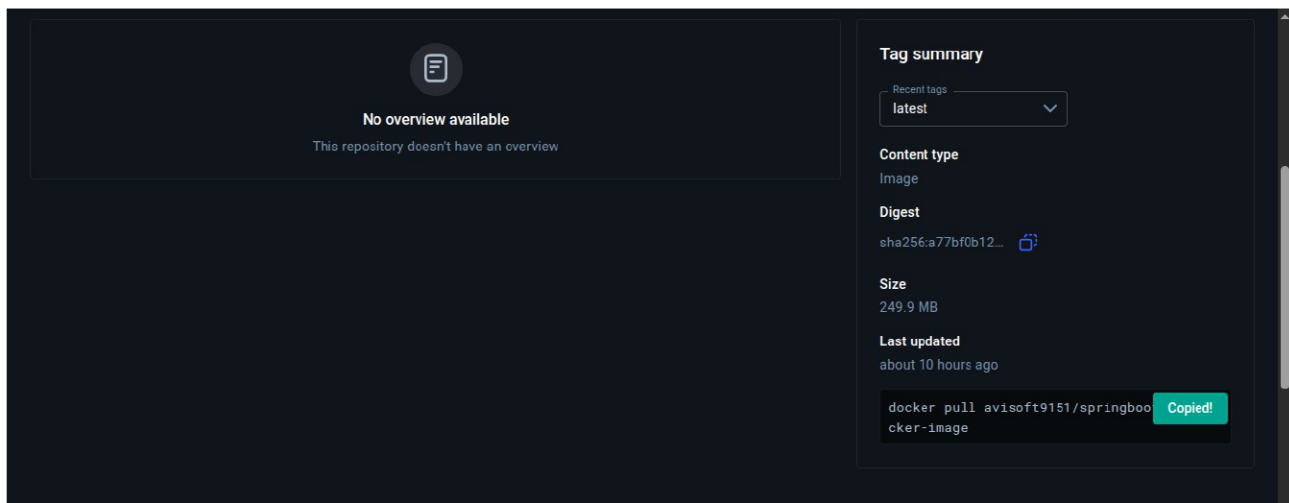
Step 24: We can pull this image and run on locally then click on **Public view**.



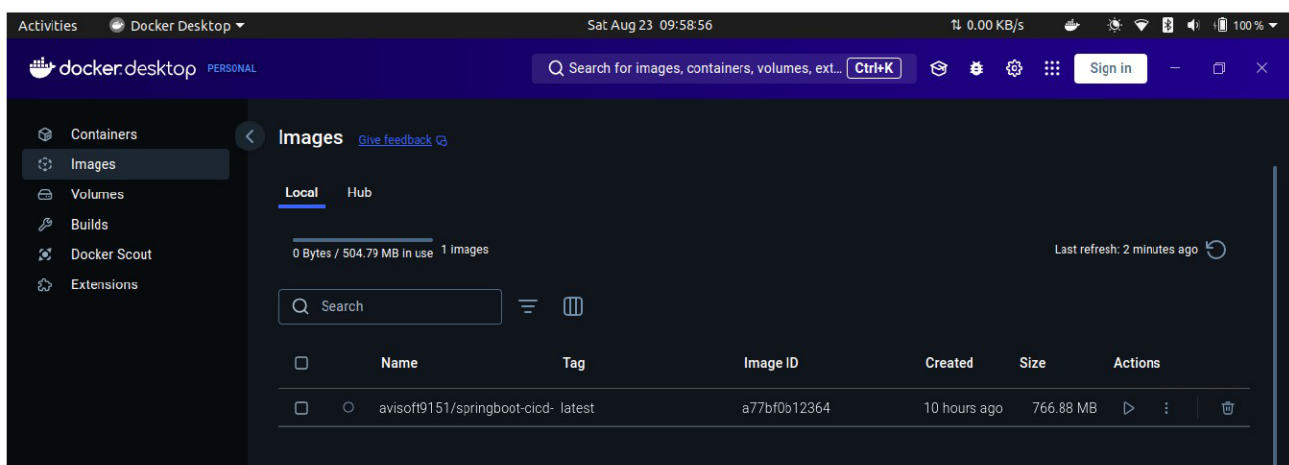
The screenshot shows the Docker Hub interface for the repository `avisoft9151/springboot-cicd-docker-image` with the 'General' tab selected. The left sidebar contains navigation links: Repositories, Collaborations, Settings, Default privacy, Notifications, Billing, Usage, Pulls, and Storage. The main content area displays the repository name, a 'Public view' button, and a table of tags. The 'Tags' tab is active, showing a list of tags with their OS, Type, Pulled, and Pushed times. The 'Docker commands' section shows the command used to push the image to the repository.

Tag	OS	Type	Pulled	Pushed
latest	linux/amd64	Image	less than 1 day	10 minutes

Copy this command and run it on terminal for local run.

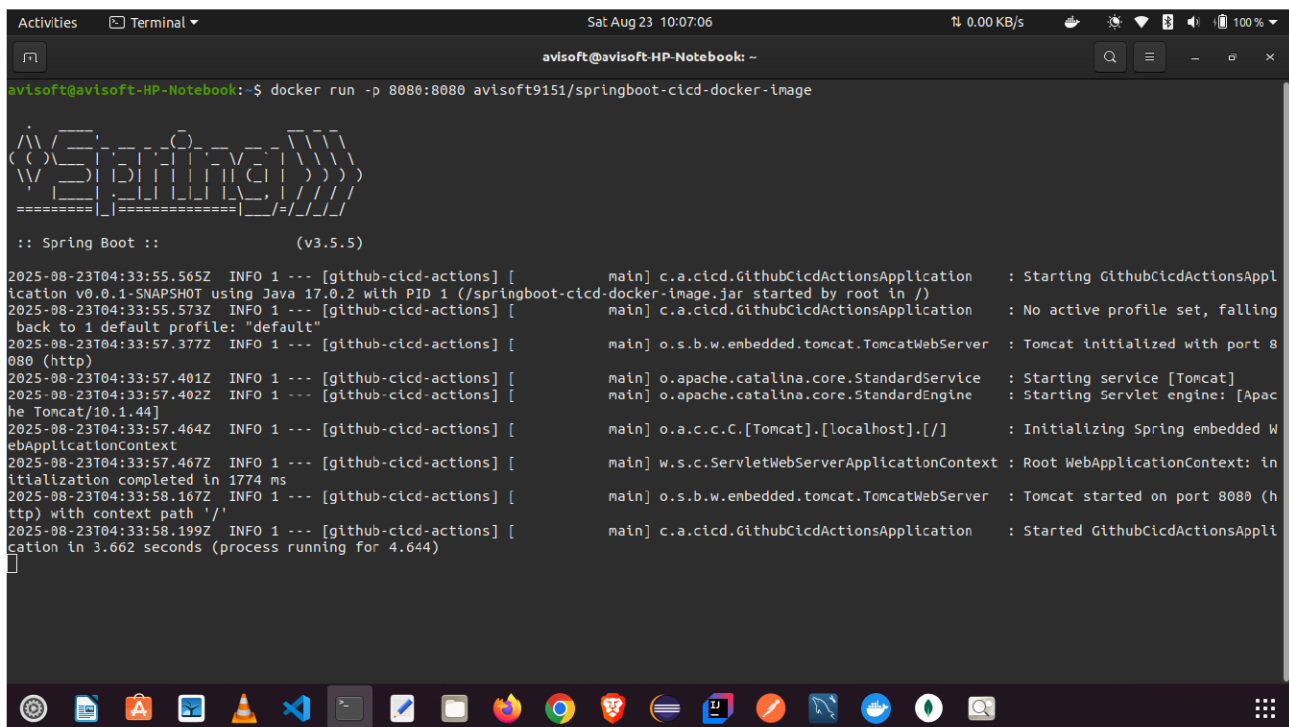


- You can see downloaded the docker image.
- And you observe the docker image add after run - `docker images` command run on terminal.
- Now open the Docker Desktop and you can see this image added.



Now we run this image manually through command and run this command on terminal.

Command - [docker run -p 8080:8080 avisoft9151/springboot-cicd-docker-image](#)

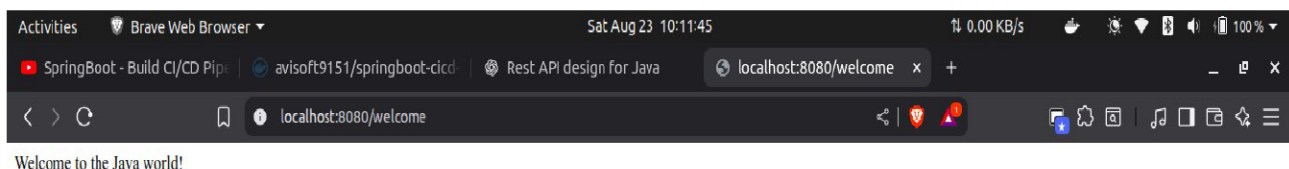


```
avisoft@avisoft-HP-Notebook:~$ docker run -p 8080:8080 avisoft9151/springboot-cicd-docker-image

:: Spring Boot ::
(v3.5.5)

2025-08-23T04:33:55.565Z INFO 1 --- [github-cicd-actions] [main] c.a.cicd.GithubCicdActionsApplication : Starting GithubCicdActionsAppl
ication v0.0.1-SNAPSHOT using Java 17.0.2 with PID 1 (/springboot-cicd-docker-image.jar started by root in /)
2025-08-23T04:33:55.573Z INFO 1 --- [github-cicd-actions] [main] c.a.cicd.GithubCicdActionsApplication : No active profile set, falling
back to 1 default profile: "default"
2025-08-23T04:33:57.377Z INFO 1 --- [github-cicd-actions] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8
080 (http)
2025-08-23T04:33:57.401Z INFO 1 --- [github-cicd-actions] [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2025-08-23T04:33:57.402Z INFO 1 --- [github-cicd-actions] [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apac
he Tomcat/10.1.44]
2025-08-23T04:33:57.464Z INFO 1 --- [github-cicd-actions] [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded W
ebApplicationContext
2025-08-23T04:33:57.467Z INFO 1 --- [github-cicd-actions] [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: in
itialization completed in 1774 ms
2025-08-23T04:33:58.167Z INFO 1 --- [github-cicd-actions] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (h
ttp) with context path '/'
2025-08-23T04:33:58.199Z INFO 1 --- [github-cicd-actions] [main] c.a.cicd.GithubCicdActionsApplication : Started GithubCicdActionsAppli
cation in 3.662 seconds (process running for 4.644)
```

- We can see Application started at port number 8080 and docker container.
- Now go to the browser and hit this endpoints - <http://localhost:8080/welcome>



Final Outcome

- **Fully automated CI/CD pipeline:**
 - Code push → GitHub Actions → Build → Docker Image → Push to Docker Hub.
- No manual build required.
- Can deploy this image anywhere (local, Kubernetes, cloud).